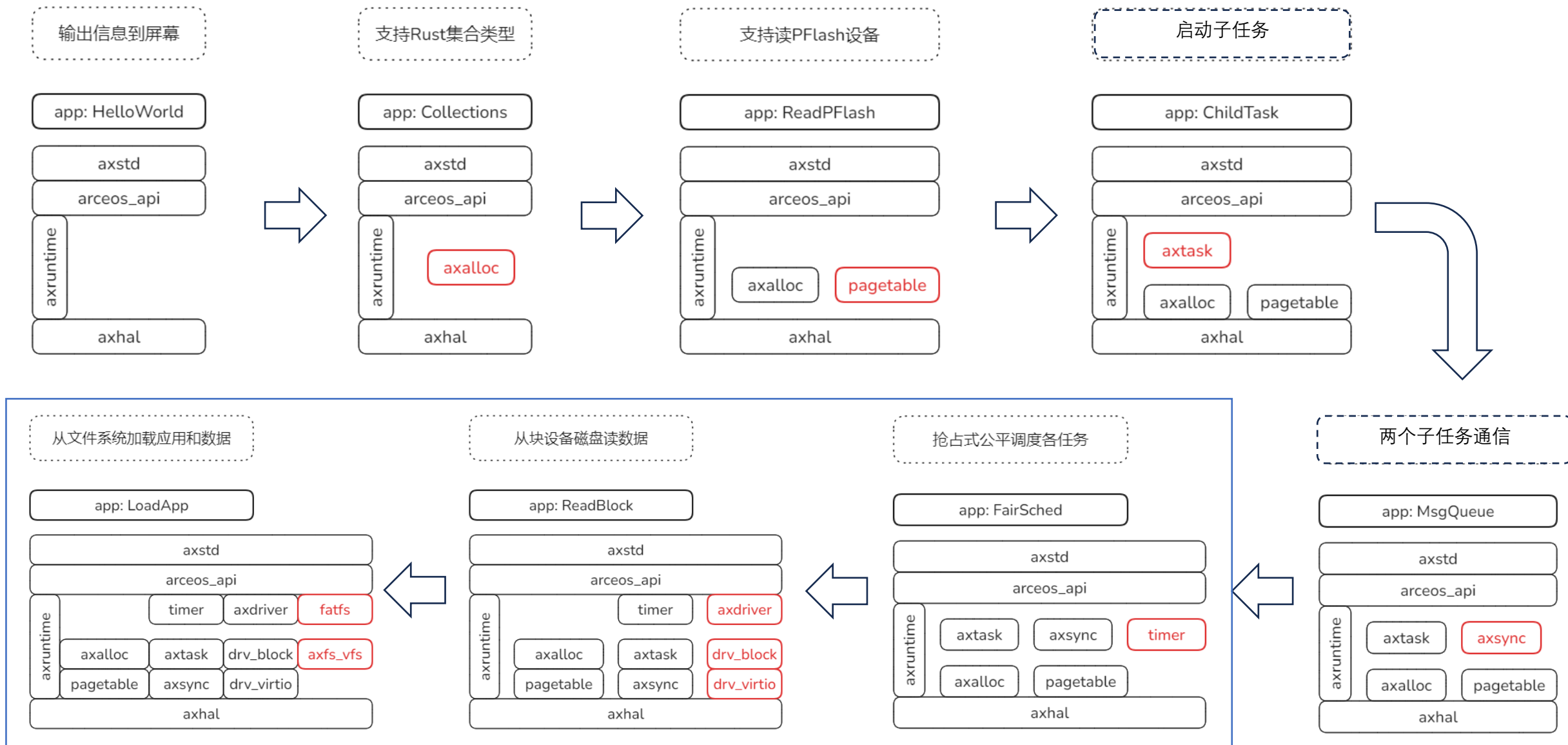


秋冬季训练营三阶段 组件化内核设计与实践(3)

清华大学 & 山东乾云启创
泉城实验室安全操作系统联合创新中心
石磊

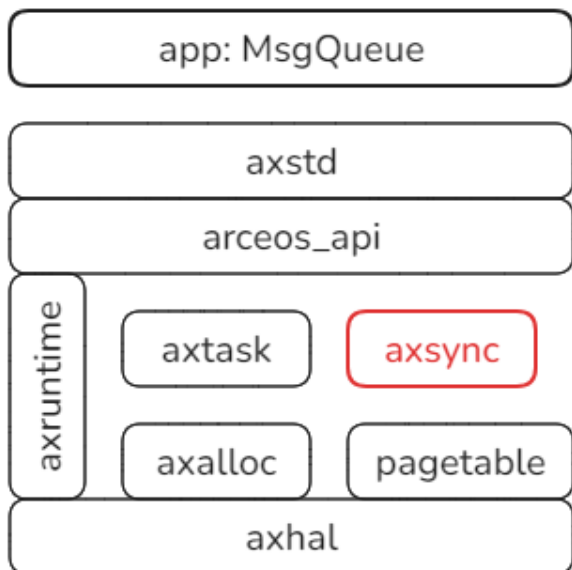
2024.11.15

第一部分 Unikernel

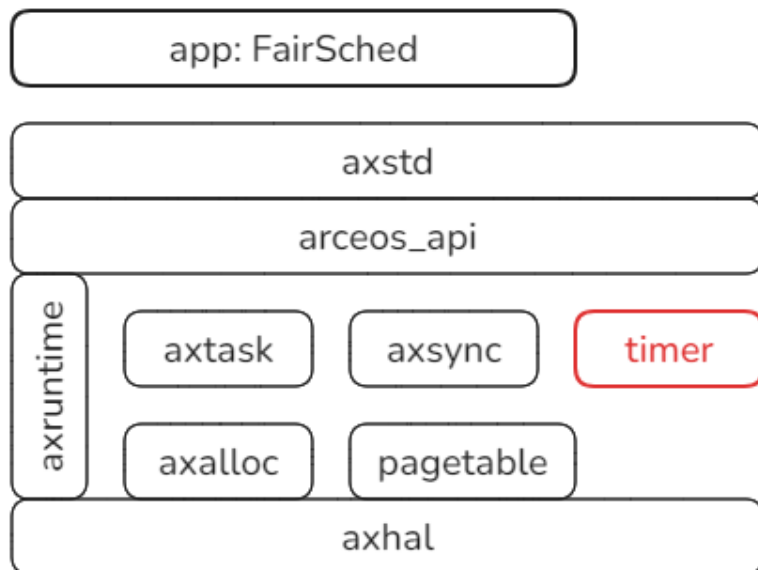


U.6.0 FairSched

父子任务基于加锁队列通信



抢占式公平调度各任务



```
[dependencies]  
axstd = { features = ["alloc", "paging", "multitask",
```

实验命令行:

make run A=tour/u_6_0

make run A=tour/u_6_1

两个实验都能观察到:
任务执行过程中被抢占

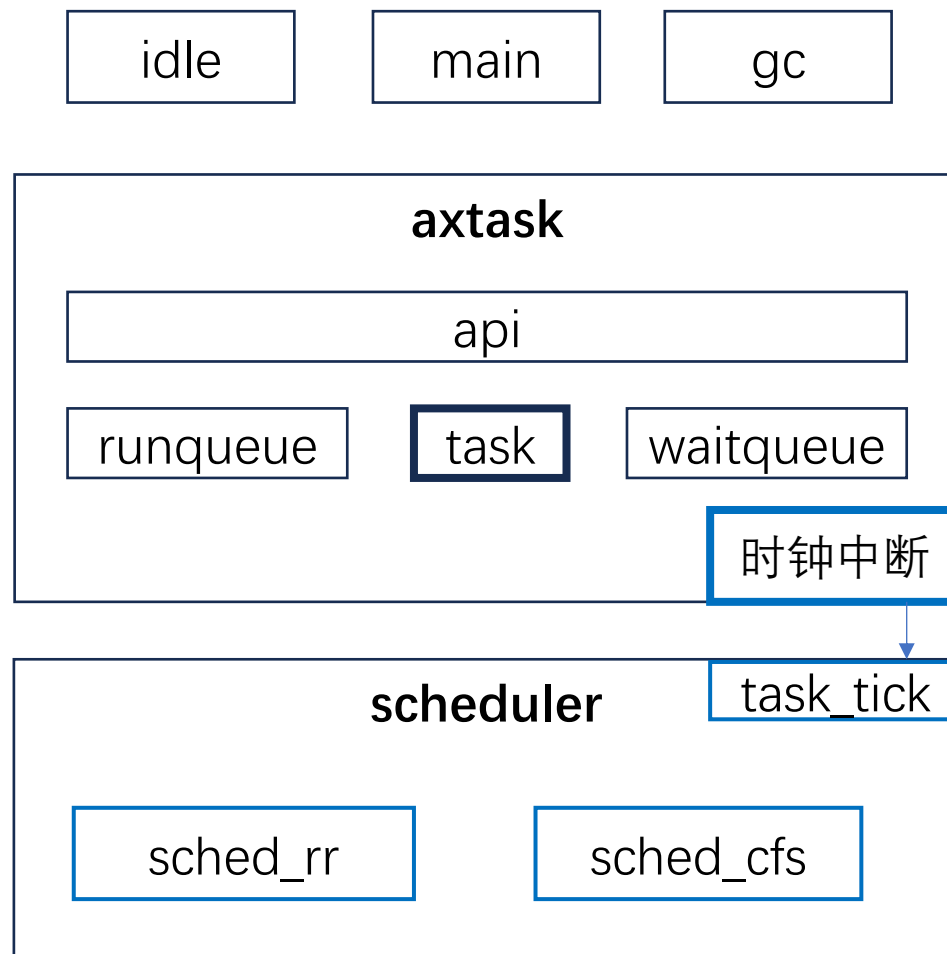
本节目标:

1. 抢占式调度机制, CFS调度策略
2. 时钟中断机制

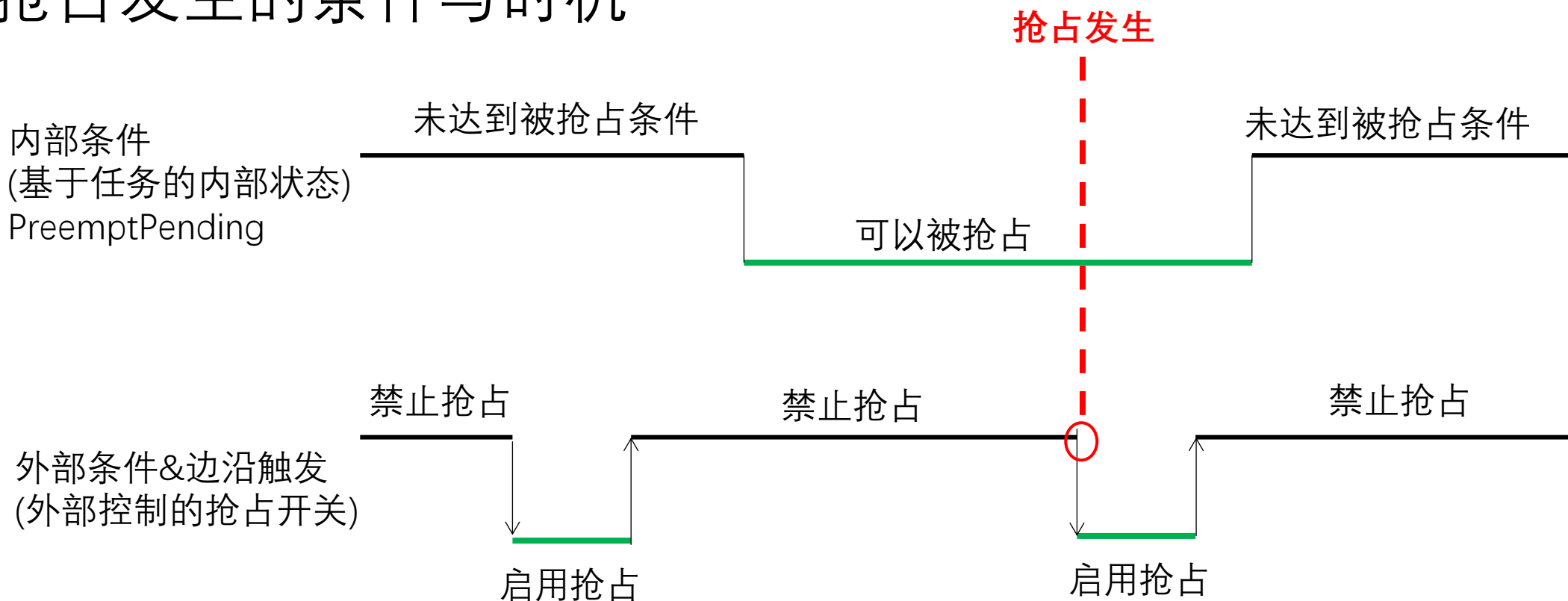
抢占式调度 – PreemptiveSched

抢占式调度：调度器依据策略，可以打断**当前任务**的执行，移交CPU执行权给当前“更”有资格的任务。
抢占机制的根本保障是系统定时器。所以抢占针对的主要操作目标就是**current task当前任务**。

机制与时机：**不是无条件**的抢占，要两个条件都具备
一是任务内部达到了某种条件，例如时间片耗尽；
二是外部条件与时机，在preempt从disable到enable的那个状态切换点触发抢占。



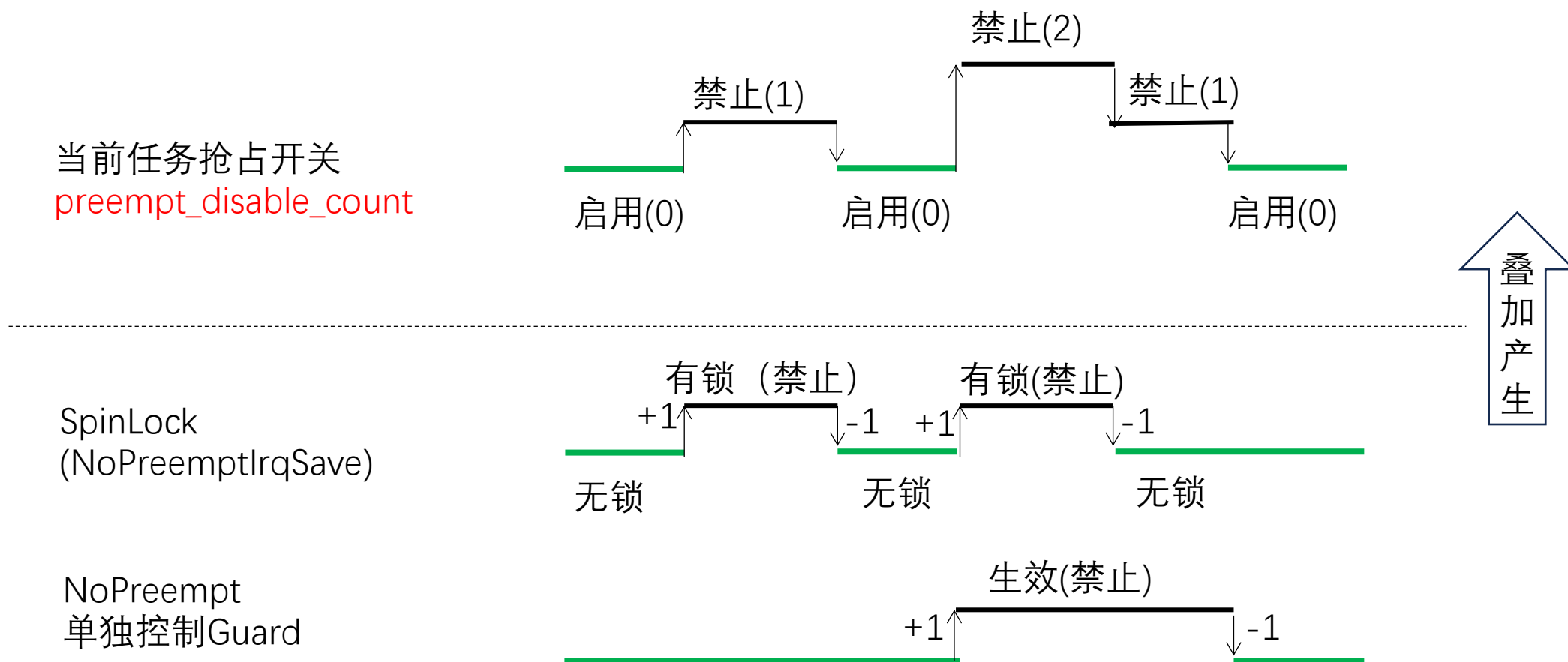
抢占发生的条件与时机



1. 只有内外条件都满足时，才发生抢占；内部条件举例任务时间片耗尽，外部条件类似定义某种临界区，控制什么时候不能抢占，本质上它基于当前任务的`preempt_disable_count`。
2. 只在 禁用->启用 切换的下边沿触发；下边沿通常在自旋锁解锁时产生，此时是切换时机。
3. 推动内部条件变化(例: 任务时间片消耗)和边沿触发产生(例: 自旋锁加解锁)的根本源是时钟中断。

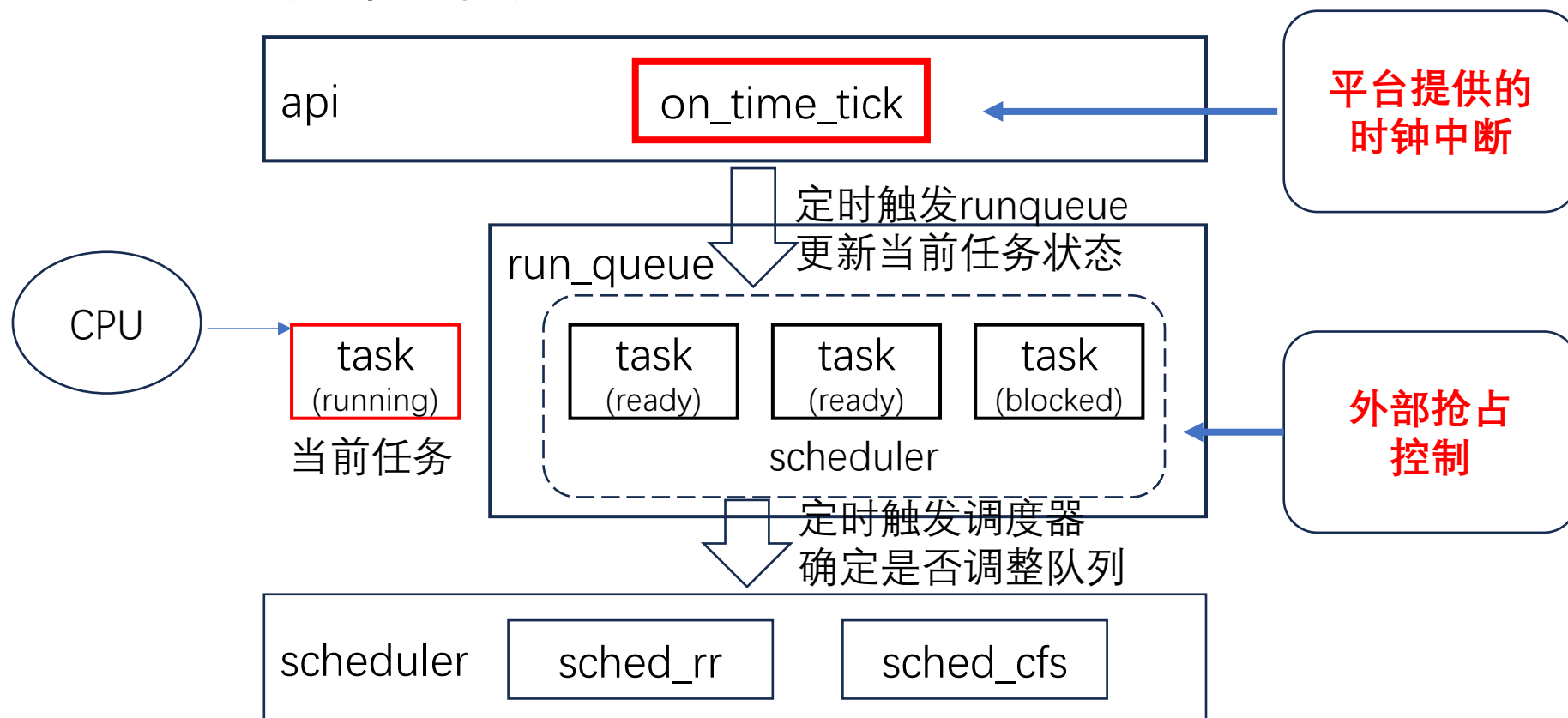
外部条件：外部控制的抢占开关(示例)

抢占针对的目标就是当前任务，由外部控制的抢占开关是当前任务的`preempt_disable_count`。
作为计数：0代表开抢占，大于0则关抢占(可叠加，所以可能大于1)



抢占式调度与协作式在框架上的关键区别

抢占式调度在协作式自主yield的基础上，引入抢占控制和时钟中断，动态调整内外条件，触发优先级高的任务及时获得调用机会，避免个别任务长期不合理的占据CPU。



时钟中断与抢占式调度

通过axhal 注册时钟中断，定期触发
axtask::on_timer_tick



经由axtask::runqueue传递定时事件



触发特定调度器的task_tick，决定是否
标记抢占标志，并可能进一步的导
致任务队列的重排

```
#[cfg(feature = "irq")]
fn init_interrupt() {
    use axhal::time::TIMER_IRQ_NUM;
    axhal::irq::register_handler(TIMER_IRQ_NUM, || {
        update_timer();
        #[cfg(feature = "multitask")]
        axtask::on_timer_tick();
    });

    // Enable IRQs before starting app
    axhal::arch::enable_irqs();
}
```

```
#[cfg(feature = "irq")]
#[doc(cfg(feature = "irq"))]
pub fn on_timer_tick() {
    crate::timers::check_events();
    RUN_QUEUE.lock().scheduler_timer_tick();
}
```

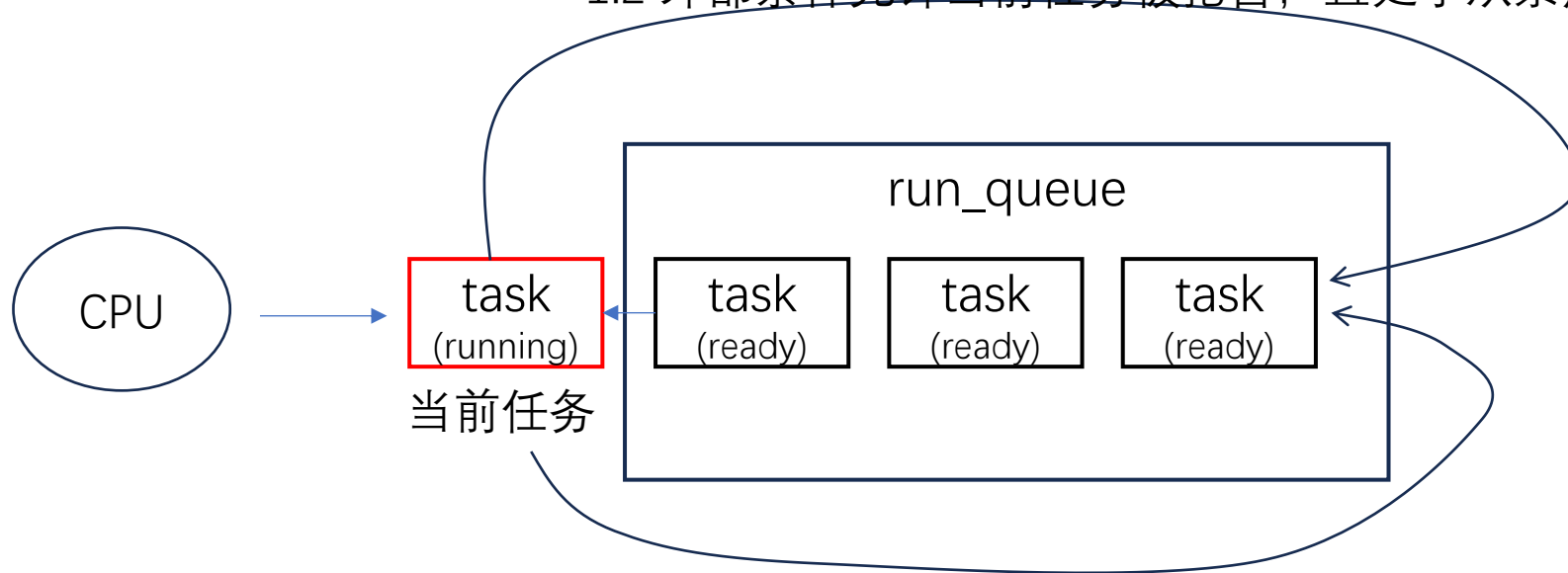
```
#[cfg(feature = "irq")]
pub fn scheduler_timer_tick(&mut self) {
    let curr = crate::current();
    if !curr.is_idle() && self.scheduler.task_tick(curr.as_task_ref()) {
        #[cfg(feature = "preempt")]
        curr.set_preempt_pending(true);
    }
}
```


抢占式调度算法ROUND_ROBIN机制

在协作式调度FIFO的基础上，由定时器定时递减**当前任务**的时间片，耗尽时允许调度，一旦外部条件符合，边沿触发抢占，当前任务排到队尾，如此完成各个任务的循环排列。注：核心目标是**当前任务**。

抢占式：同时满足以下条件

- 1.1 定时器递减当前任务的时间片计数，减到0时，设**preempt pending**
- 1.2 外部条件允许当前任务被抢占，且处于从禁用到启用的边沿



2 仍继承协作式：主动执行yield将会排到队尾

抢占式调度算法ROUND_ROBIN实现

```
pub struct RRTask<T, const MAX_TIME_SLICE: usize> {  
    inner: T,  
    time_slice: AtomicIsz,ze,  
}
```

任务维护时间片作为本轮生命,
耗尽时将**有可能**被调度出去

```
pub struct RRScheduler<T, const MAX_TIME_SLICE: usize> {  
    ready_queue: VecDeque<Arc<RRTask<T, MAX_TIME_SLICE>>>,  
}
```

调度器队列是一个双端可操作的队列

```
fn pick_next_task(&mut self) -> Option<Self::SchedItem> {  
    self.ready_queue.pop_front()  
}  
  
fn put_prev_task(&mut self, prev: Self::SchedItem, preempt: bool) {  
    if prev.time_slice() > 0 && preempt {  
        self.ready_queue.push_front(prev)  
    } else {  
        prev.reset_time_slice();  
        self.ready_queue.push_back(prev)  
    }  
}  
  
fn task_tick(&mut self, current: &Self::SchedItem) -> bool {  
    let old_slice = current.time_slice.fetch_sub(1, Ordering::Release);  
    old_slice <= 1  
}
```

时间片耗尽时, 放到队尾, 即调度出去;
否则, 保存队首, 即仍是当前任务。

关键: 由时钟中断定时触发, 每次递减;
接近到0时, 内部被抢占条件具备, 返回
true表示可以被抢占。

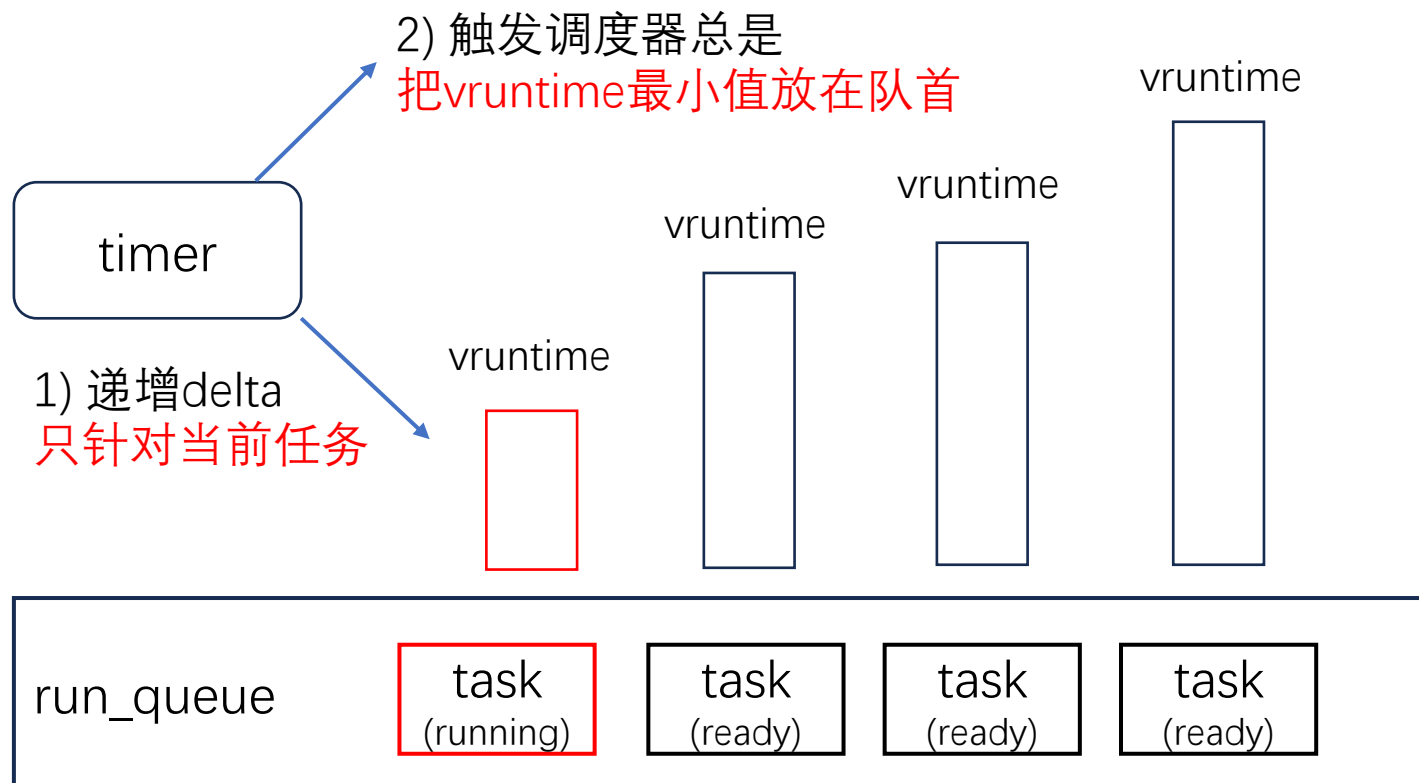
抢占式调度算法CFS(Completely Fair Scheduler)

vruntime最小的任务就是优先权最高任务，即**当前任务**。

计算公式：

$$\text{vruntime} = \text{init_vruntime} + (\text{delta} / \text{weight}(\text{nice}))$$

系统初始化时，init_vruntime, delta, nice三者都是0



新增任务：

新任务的init_vruntime等于min_vruntime
即默认情况下新任务能够尽快投入运行

设置优先级set_priority：

只有CFS支持设置优先级，即nice值，会影响init_vruntime以及运行中vruntime值，该任务会比默认情况获得更多或更少的运行机会。

抢占式调度算法CFS(Completely Fair Scheduler)

```
pub struct CFScheduler<T> {  
    ready_queue: BTreeMap<(isize, isize), Arc<CFSTask<T>>>,  
    min_vruntime: Option<AtomicIsize>,  
    id_pool: AtomicIsize,  
}
```

任务队列基于BtreeMap，即有序队列，排序基于vruntime

```
fn pick_next_task(&mut self) -> Option<Self::SchedItem> {  
    if let Some((_, v)) = self.ready_queue.pop_first() {  
        Some(v)  
    } else {  
        None  
    }  
}
```

队首永远是vruntime最小的任务，即当前应该被调度的目标任务

```
fn put_prev_task(&mut self, prev: Self::SchedItem, _preempt: bool) {  
    let taskid = self.id_pool.fetch_add(1, Ordering::Release);  
    prev.set_id(taskid);  
    self.ready_queue  
        .insert((prev.clone().get_vruntime(), taskid), prev);  
}
```

由于当前任务执行后，vruntime必然会发生变化，必须重新计算插入适当位置。

```
fn task_tick(&mut self, current: &Self::SchedItem) -> bool {  
    current.task_tick();  
    self.min_vruntime.is_none()  
        || current.get_vruntime() > self.min_vruntime.as_mut().unwrap()  
}
```

更新**当前任务**的vruntime;
首次运行，或者当前任务vruntime比min_vruntime大，就会触发resched

抢占式调度算法CFS(Completely Fair Scheduler)

```
pub struct CFSTask<T> {  
    inner: T,  
    init_vruntime: AtomicIsz,   
    delta: AtomicIsz,   
    nice: AtomicIsz,   
    id: AtomicIsz,   
}
```

Vruntime计算公式涉及的基础参数:
init_vruntime创建时固定
delta由定时器递增
nice由set_priority设置

```
fn task_tick(&self) {  
    self.delta.fetch_add(1, Ordering::Release);  
}
```

delta由定时器递增。如前页，只有当前任务由此机会。

```
fn get_vruntime(&self) -> isize {  
    if self.nice.load(Ordering::Acquire) == 0 {  
        self.init_vruntime.load(Ordering::Acquire) + self.delta.load(Ordering::Acquire)  
    } else {  
        self.init_vruntime.load(Ordering::Acquire)  
        + self.delta.load(Ordering::Acquire) * 1024 / self.get_weight()  
    }  
}
```

Vruntime计算公式

```
fn set_priority(&self, nice: isize) {  
    let current_init_vruntime = self.get_vruntime();  
    self.init_vruntime  
        .store(current_init_vruntime, Ordering::Release);  
    self.delta.store(0, Ordering::Release);  
    self.nice.store(nice, Ordering::Release);  
}
```

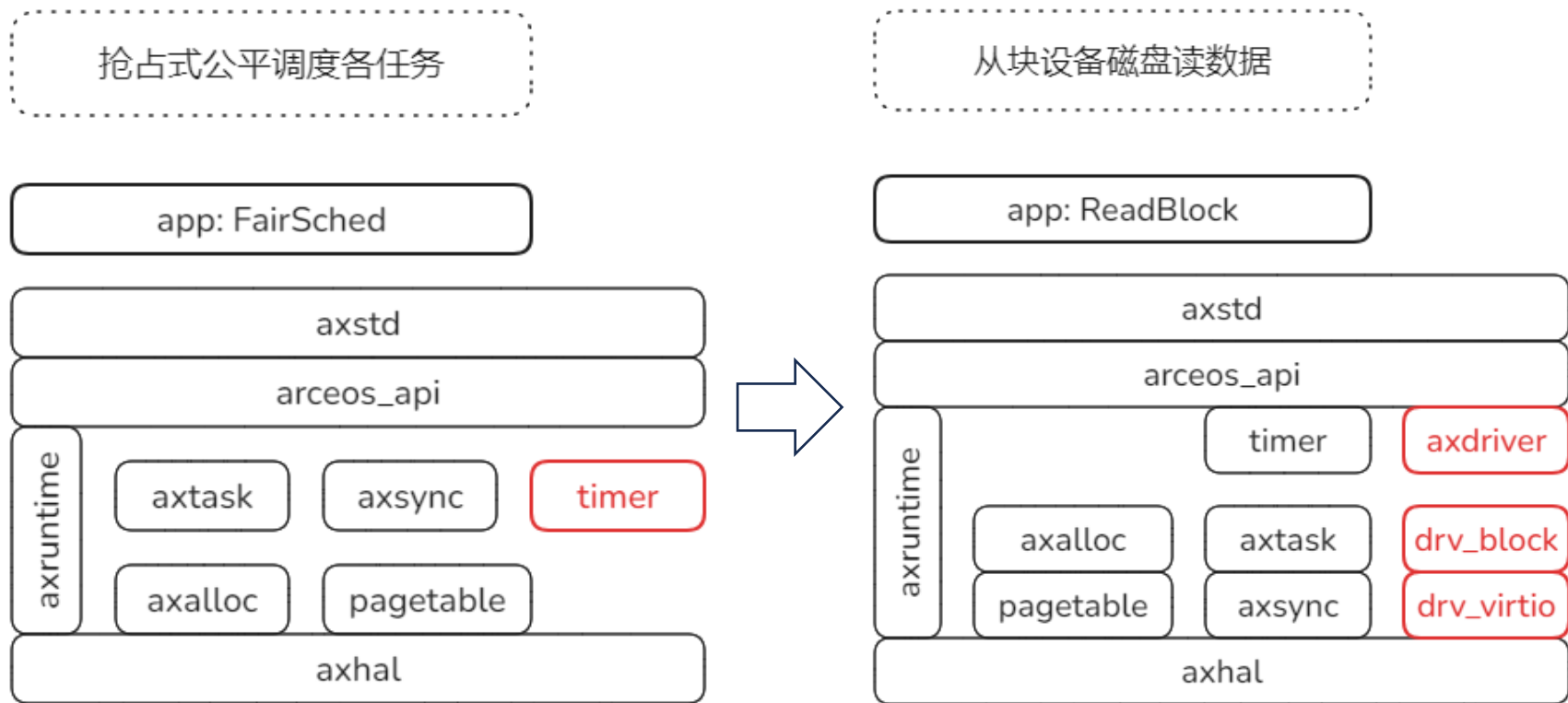
设置nice不是直接影响当前值，而是影响未来vruntime的计算结果，步进速度会变化。

小结

建立了基础的调度框架，引入三个系统任务辅助任务管理。目前可以基于最简单的协作式和抢占式调度算法，支持多任务。



U.7.0 ReadBlock



本节目标:

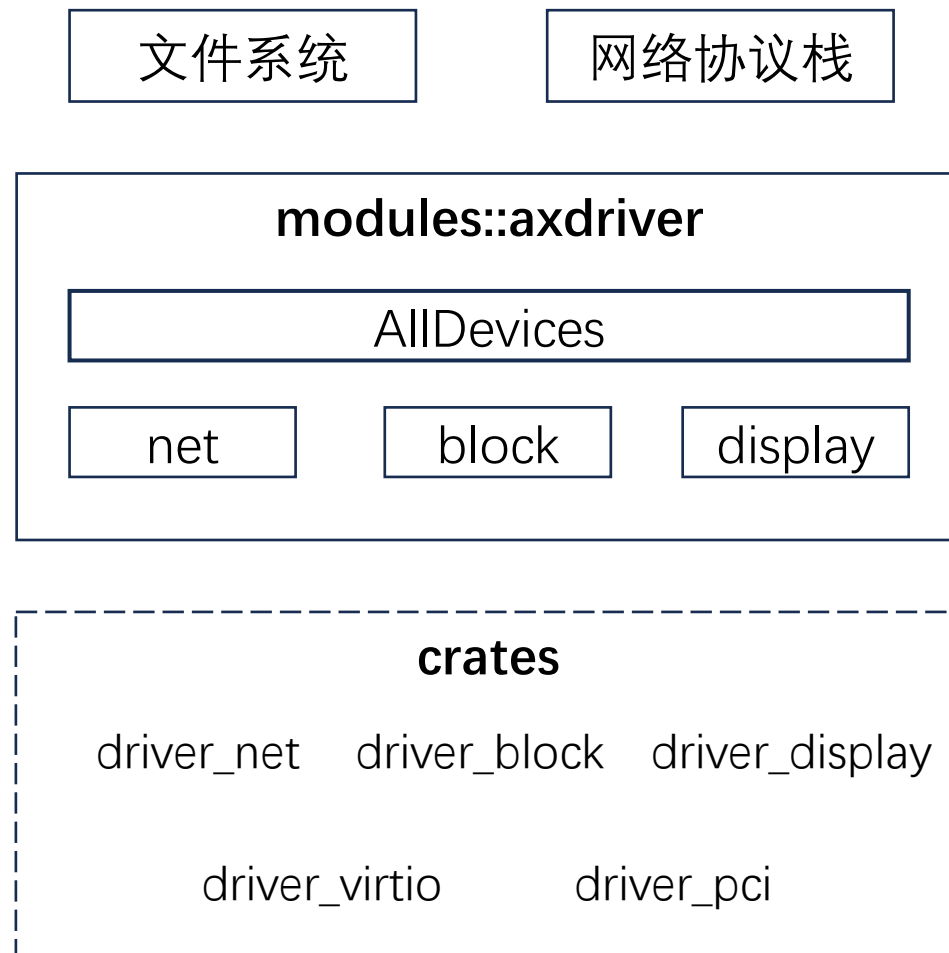
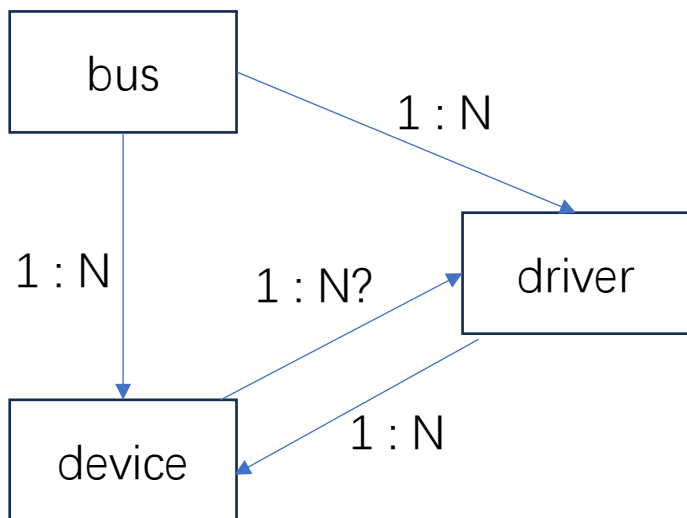
1. 从磁盘块设备读数据, 替换PFlash
2. 发现设备关联驱动、块设备、VirtIO设备

实验命令行:

`make run A=tour/u_7_0 BLK=y`

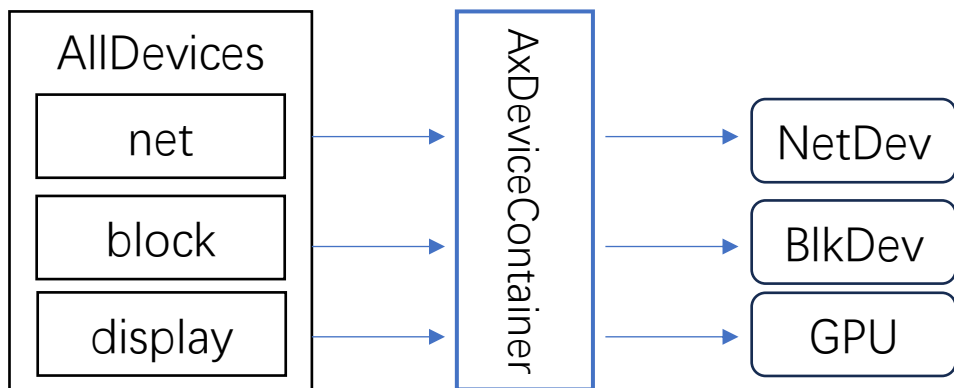
需要解决的问题

- 1) 设备管理框架
- 2) 设备发现与初始化
- 3) 中断机制与初始化



设备管理框架

AllDevices管理系统所有的设备，为上层的子系统如文件系统FS、网络协议栈NET提供访问服务。三种设备类型：



```
pub struct AllDevices {  
    /// All network device drivers.  
    #[cfg(feature = "net")]  
    pub net: AxDeviceContainer<AxNetDevice>,  
    /// All block device drivers.  
    #[cfg(feature = "block")]  
    pub block: AxDeviceContainer<AxBlockDevice>,  
    /// All graphics device drivers.  
    #[cfg(feature = "display")]  
    pub display: AxDeviceContainer<AxDisplayDevice>,  
}
```

static

```
pub struct AxDeviceContainer<D>(Option<D>);
```

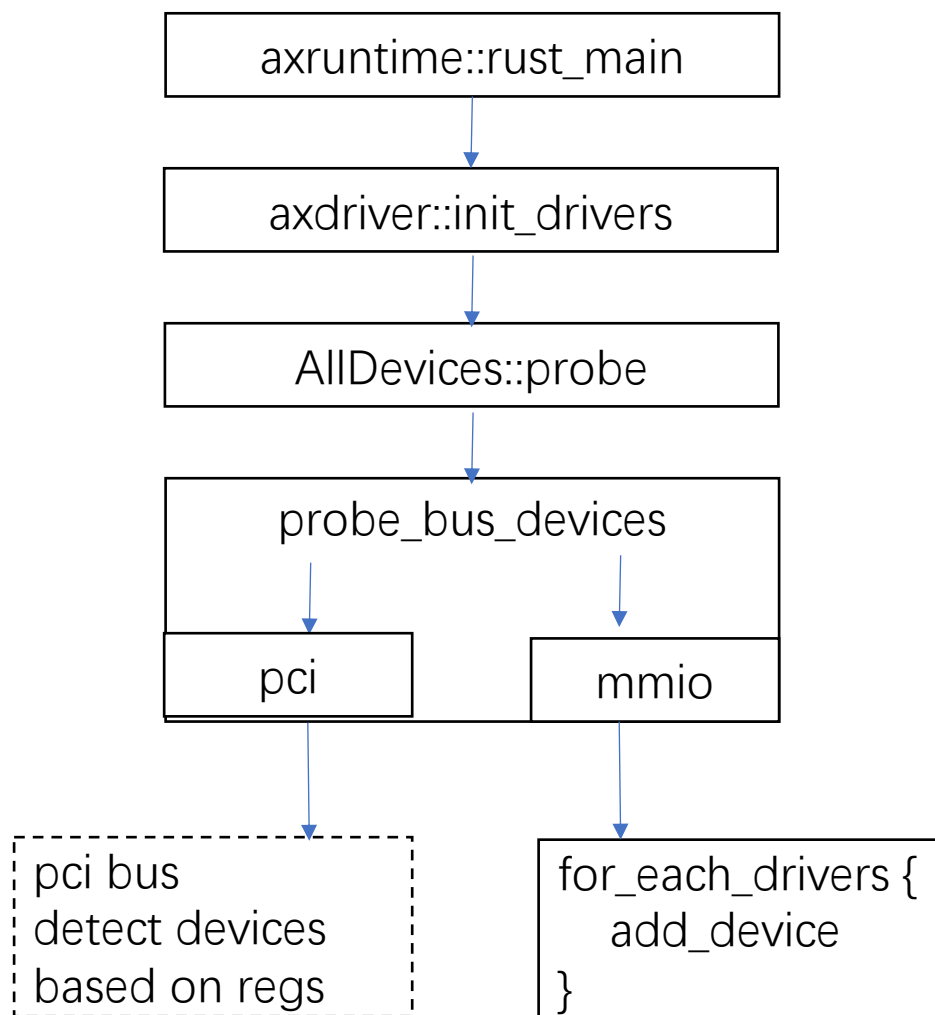
以静态方式对 具体**设备类型** 进行简单封装。
但是每种类型只能管理一个设备。
效率高

```
pub struct AxDeviceContainer<D>(Vec<D>);
```

包含一个动态可变的Vec，因而可以为每个类型
动态管理多个设备。
效率相对低

dyn

设备发现与初始化过程



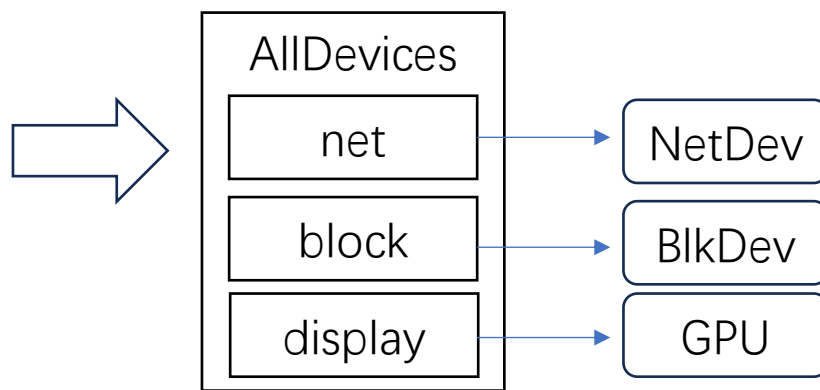
主干组件axruntime在启动后期，发现设备并用相应驱动进行初始化

axdriver负责发现设备和对其初始化的过程，核心结构AllDevices

probe基于总线发现设备，逐个匹配驱动并初始化

按照平台，有两种总线：

- 1) PCI总线：基于PCI总线协议发现和管理设备，对应PC & Server
- 2) MMIO总线：通常基于FDT解析发现和管理设备(目前未实现)



输出结果AllDevices

static: 单映射，效率高

dyn: 适用性强，效率低

基于总线发现设备- qemu平台示例

modules/axdriver/src/bus/mmio.rs

```
pub(crate) fn probe_bus_devices(&mut self) {  
    // TODO: parse device tree  
    #[cfg(feature = "virtio")]  
    for reg in axconfig::VIRTIO_MMIO_REGIONS {  
        for_each_drivers!(type Driver, {  
            if let Some(dev) = Driver::probe_mmio(reg.0, reg.1) {  
                self.add_device(dev);  
                continue; // skip to the next device  
            }  
        });  
    }  
}
```

```
macro_rules! for_each_drivers {  
    (type $drv_type:ident, $code:block) => {{  
        #[cfg(net_dev = "virtio-net")]  
        {  
            type $drv_type = <virtio::VirtIoNet as VirtIoDevMeta>::Driver;  
            $code  
        }  
        #[cfg(block_dev = "virtio-blk")]  
        {  
            type $drv_type = <virtio::VirtIoBlk as VirtIoDevMeta>::Driver;  
            $code  
        }  
        #[cfg(display_dev = "virtio-gpu")]  
        {  
            type $drv_type = <virtio::VirtIoGpu as VirtIoDevMeta>::Driver;  
            $code  
        }  
    }}  
}
```

目前管理设备和驱动数量少，采用简单方式，两级循环探测发现设备：

第一级：遍历所有virtio_mmio地址范围，由平台物理内存布局决定并进行分页映射

第二级：用for_each_drivers宏枚举设备，然后对每个virtio设备probe_mmio进行探查

for_each_drivers宏：
对所有涉及的virtio设备类型，进行枚举

下步会将这个宏及上级循环用通用方式替换：
parse FDT设备树文件的方式

virtio设备的probe过程

1) qemu模拟器基于命令行产生设备

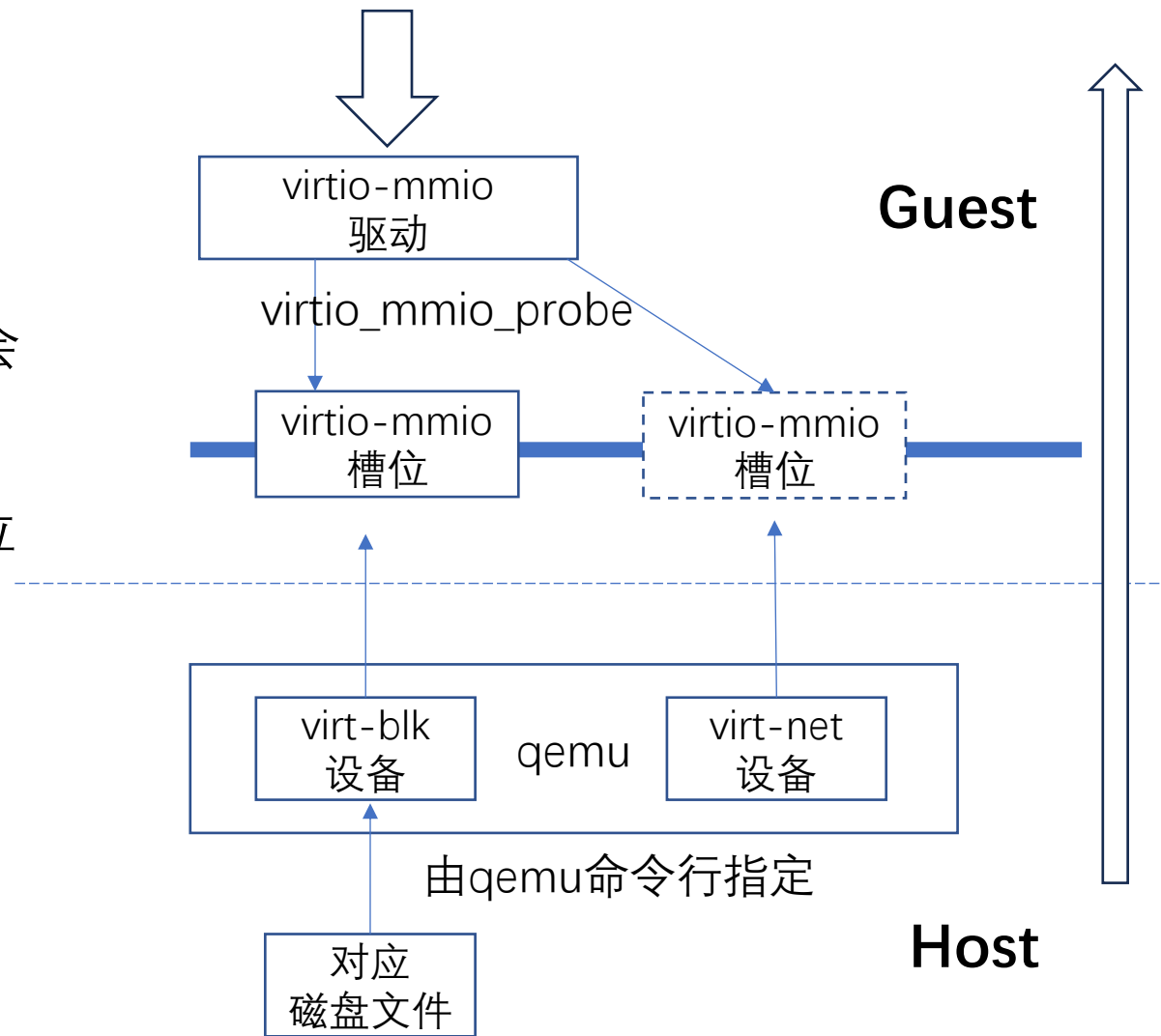
```
-device virtio-blk-device,drive=disk0  
-drive id=disk0,format=raw,file=disk.img
```

2) qemu将设备mmio地址区域映射到Guest中
qemu-virt平台默认有8个区域槽位，通常只有部分会形成映射，其它处于未映射状态，即表现为空设备

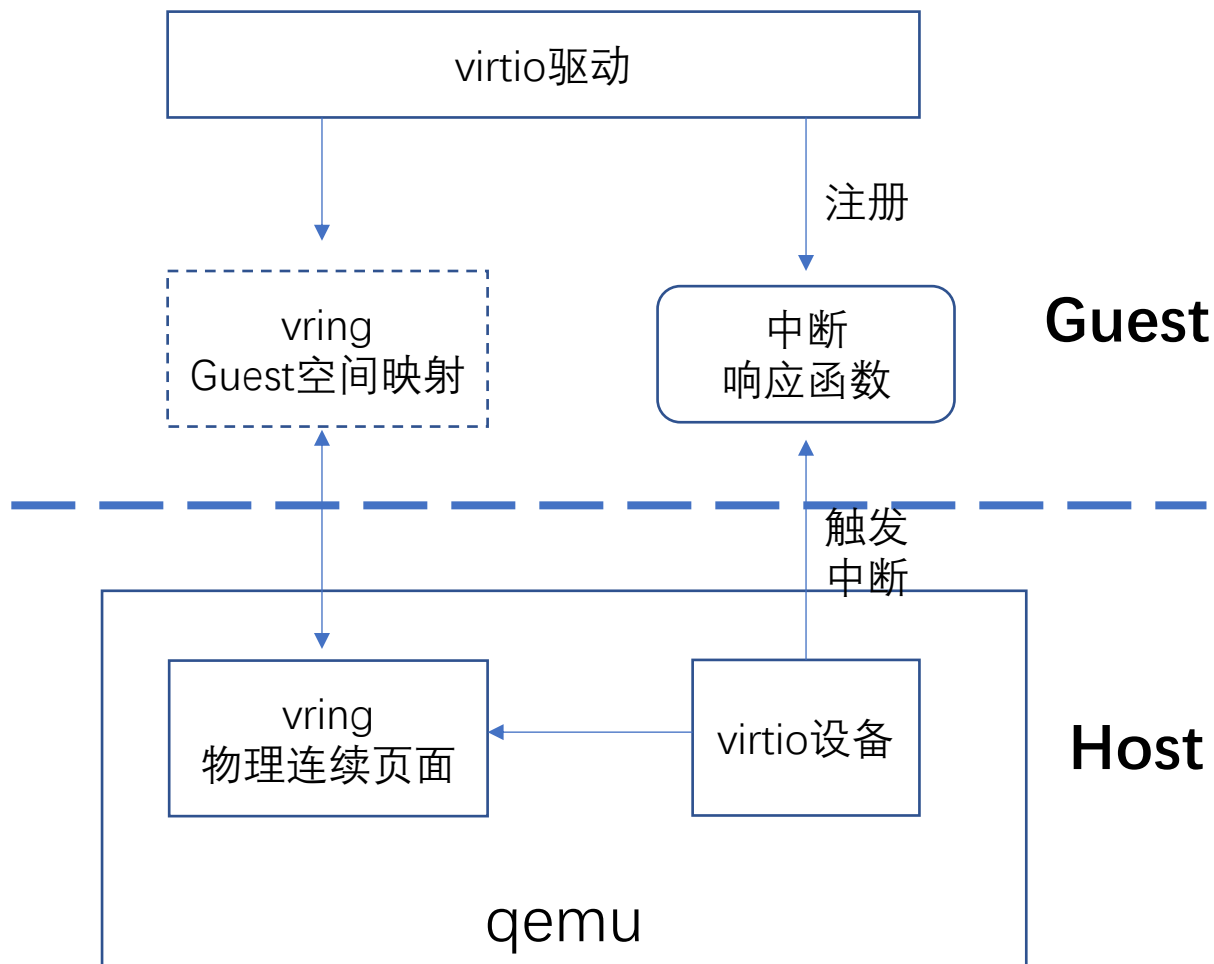
3) virtio-mmio驱动逐个发请求区探查这些区域槽位
对应映射设备响应请求，返回本设备的类型ID；
没有映射的槽位返回零，表示空设备。

4) virtio-mmio驱动把probe结果报告上层

通过virtio-mmio驱动探查发现设备



virtio驱动基本模型

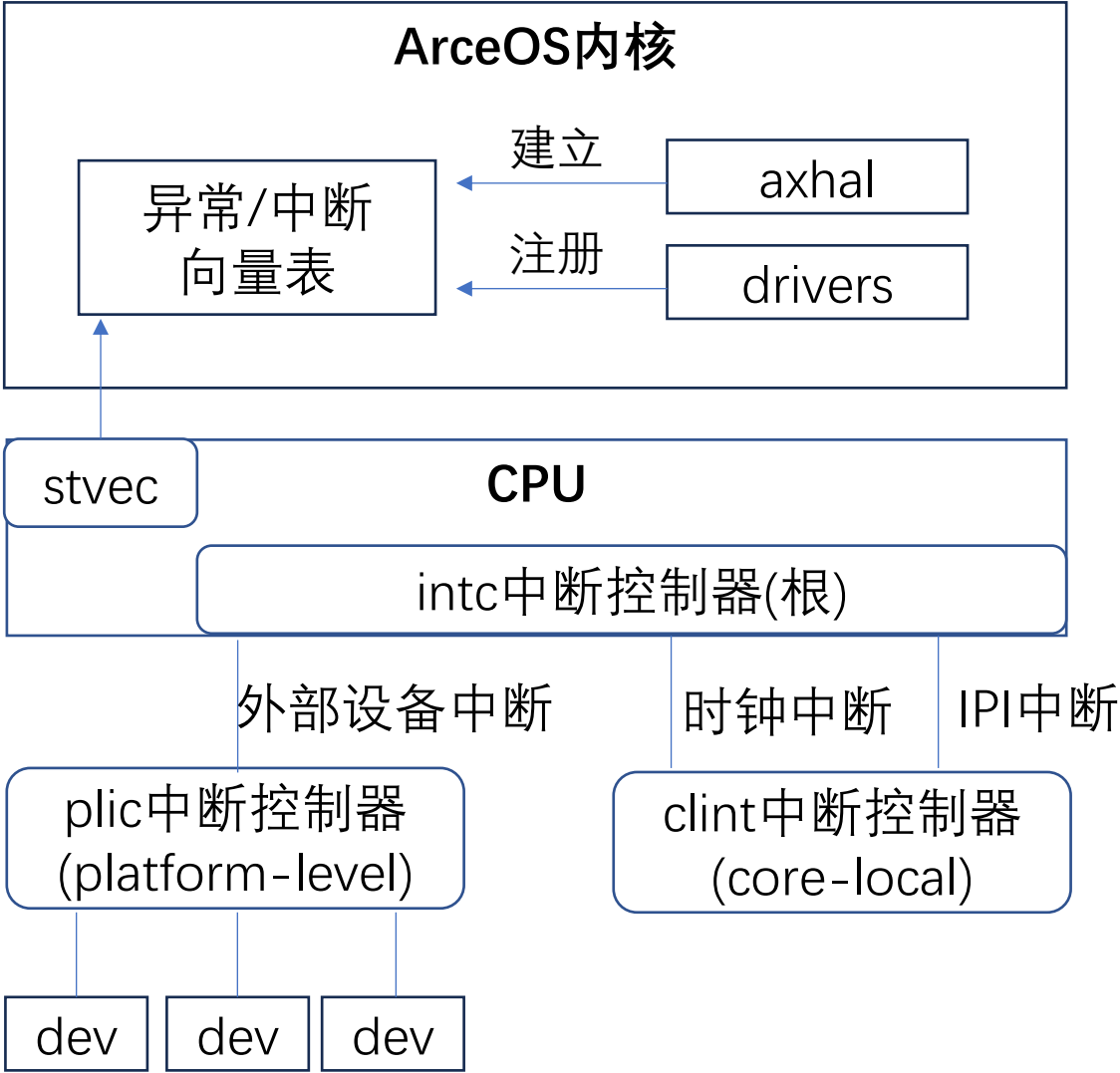


virtio驱动和virtio设备交互的两条路:

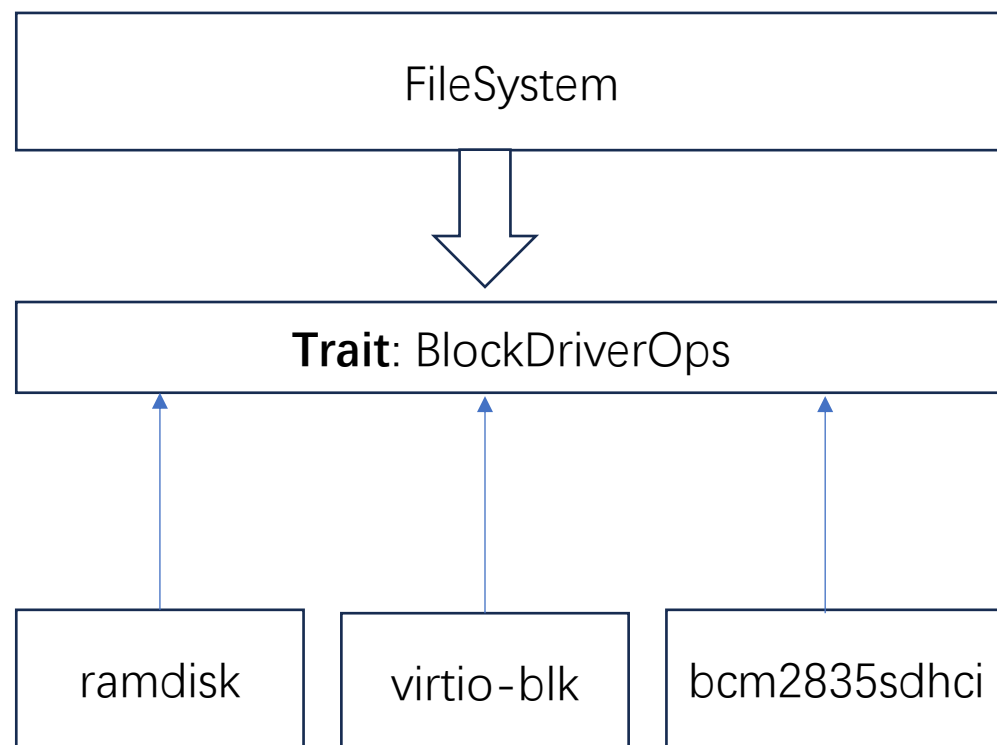
(1) 主要基于vring环形队列:
本质上是连续的Page页面, 在Guest和Host都可见可写

(2) 中断响应的通道
主要对等待读取大块数据时是有用。

中断机制与初始化(以riscv64为例)



块设备驱动的组件构成



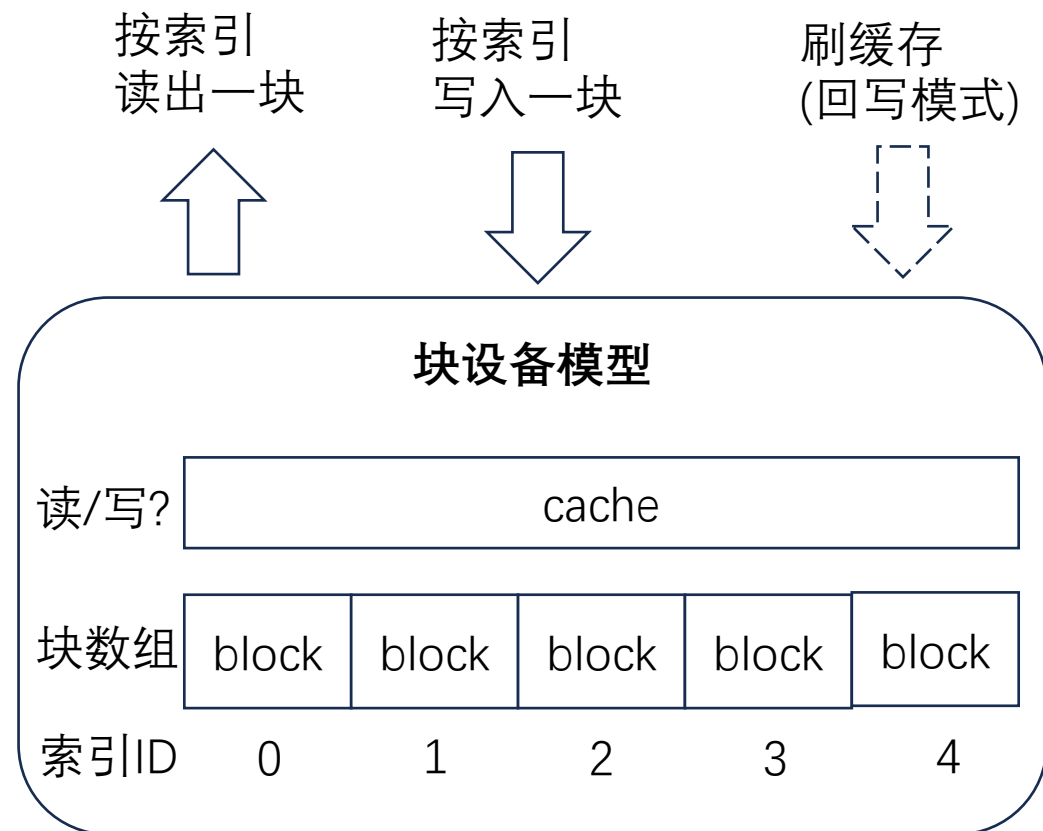
块设备驱动Trait - BlockDriverOps

```
/// Operations that require a block storage device driver to implement.
pub trait BlockDriverOps: BaseDriverOps {
    /// The number of blocks in this storage device.
    ///
    /// The total size of the device is `num_blocks() * block_size()`.
    fn num_blocks(&self) -> u64;
    /// The size of each block in bytes.
    fn block_size(&self) -> usize;

    /// Reads blocked data from the given block.
    ///
    /// The size of the buffer may exceed the block size, in which case m
    /// contiguous blocks will be read.
    fn read_block(&mut self, block_id: u64, buf: &mut [u8]) -> DevResult;

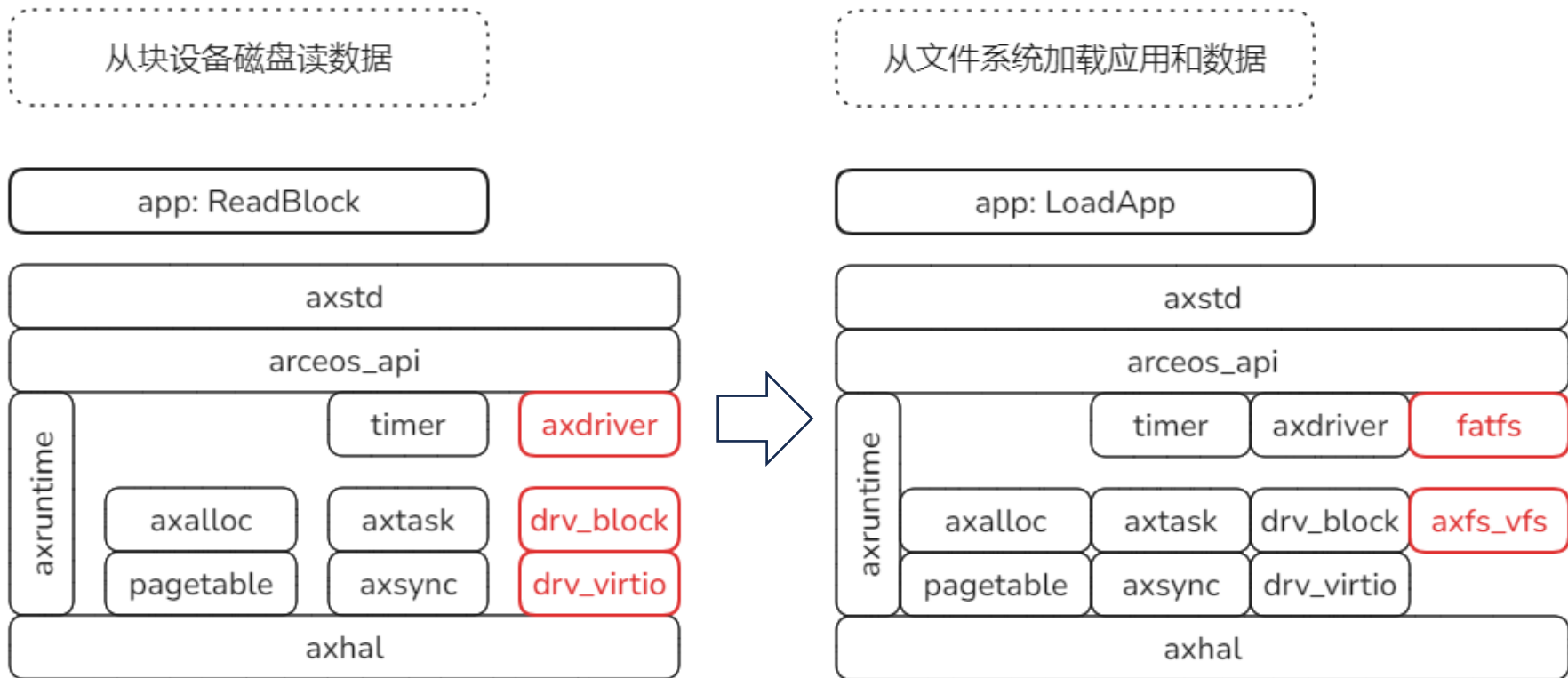
    /// Writes blocked data to the given block.
    ///
    /// The size of the buffer may exceed the block size, in which case m
    /// contiguous blocks will be written.
    fn write_block(&mut self, block_id: u64, buf: &[u8]) -> DevResult;

    /// Flushes the device to write all pending data to the storage.
    fn flush(&mut self) -> DevResult;
}
```



块设备：连续空间按指定尺寸划分为数组形式
关键属性：块大小 和 总块数

U.8.0 LoadApp



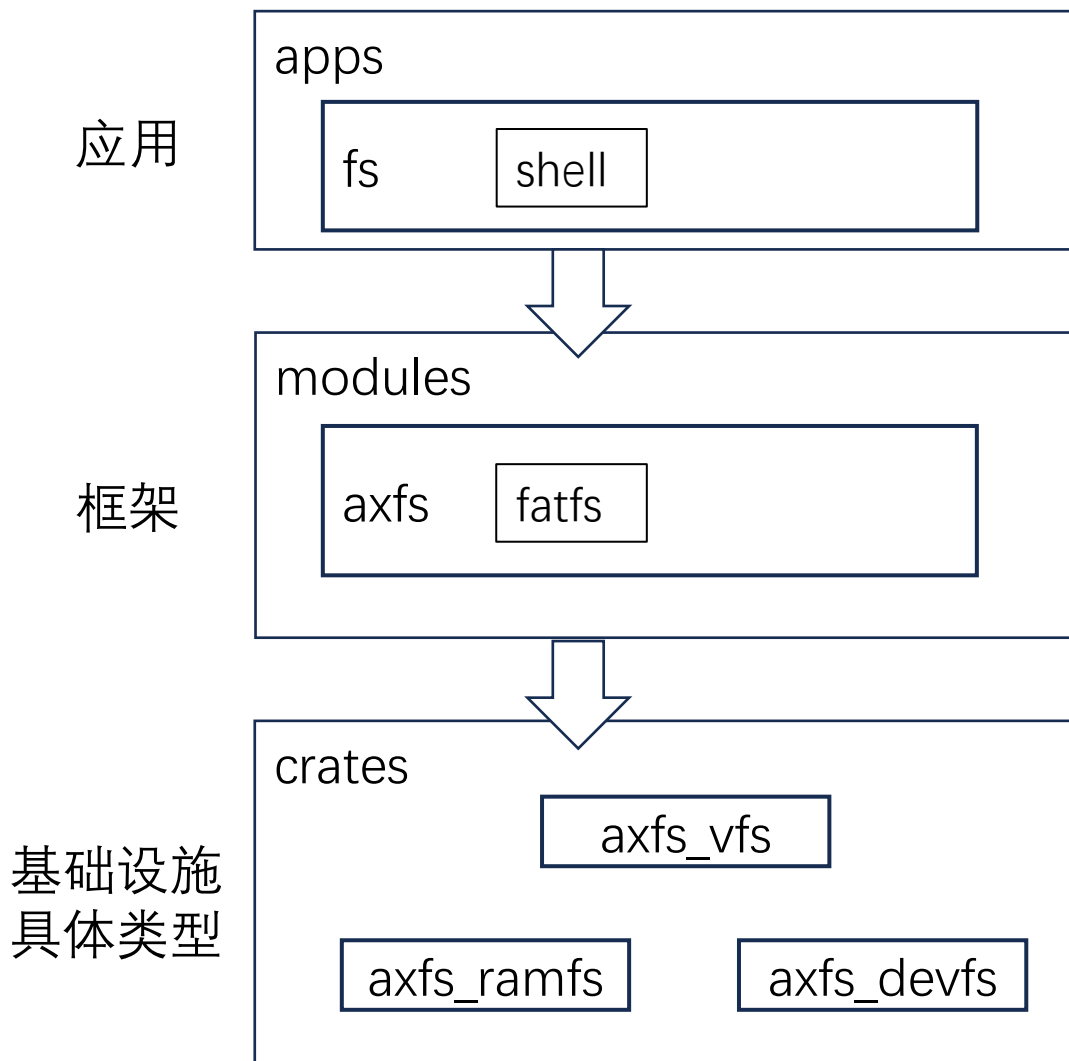
本节目标:

1. 从文件系统加载应用和数据
2. 文件系统的初始化和文件操作

实验命令行:

make run A=tour/u_8_0 BLK=y

文件系统 - 组件构成



应用shell本身模拟linux的shell
包含了一系列操作文件的子命令

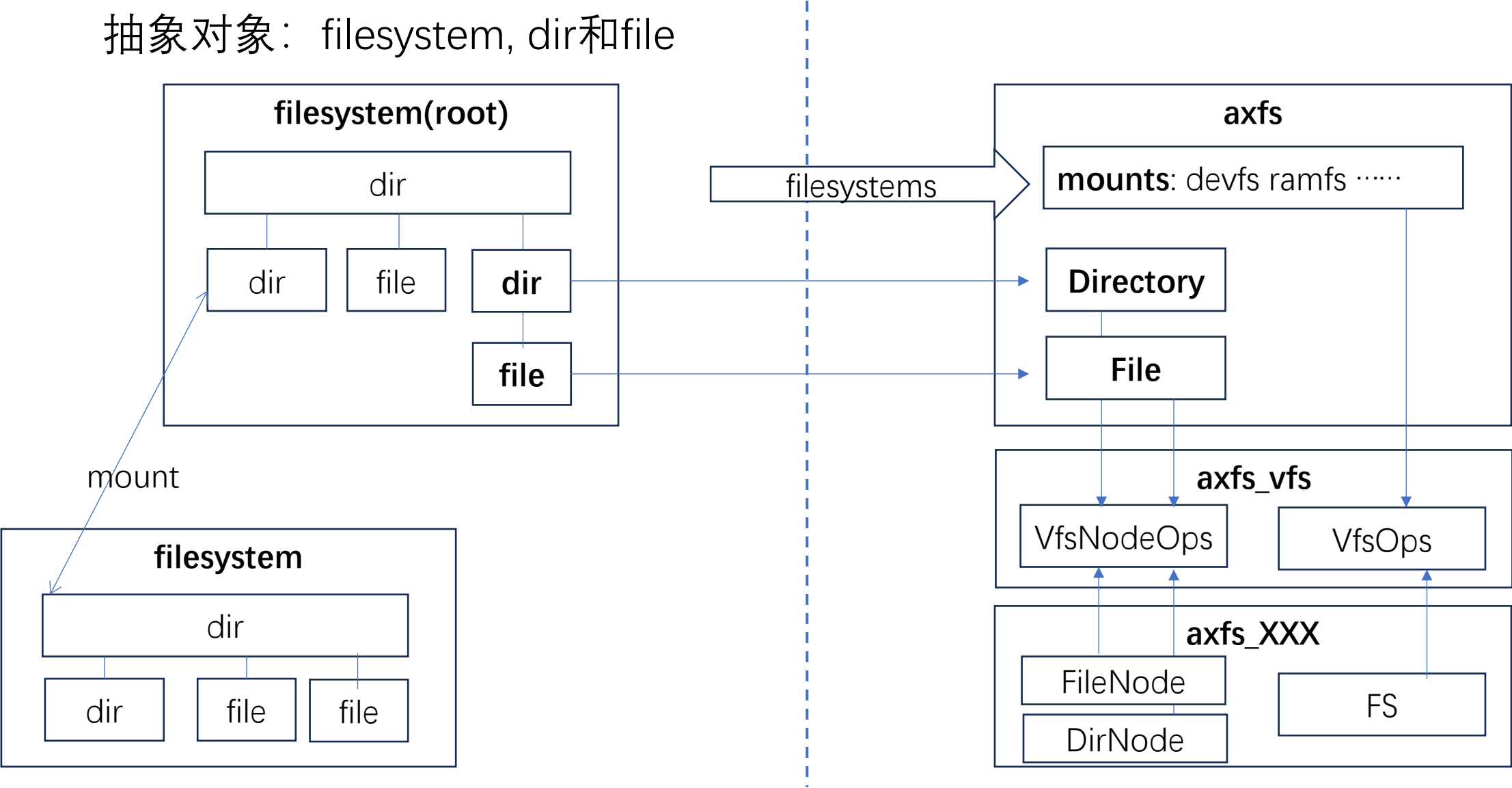
框架负责在启动时建立类似linux文件系统，
在根目录下包含普通目录与文件，及dev目录
axfs对应的是通用的目录和文件对象

VFS定义文件系统的接口层

具体的FS实现：ramfs和devfs

文件系统的抽象与对应数据结构

抽象对象： filesystem, dir和file



文件系统节点的操作流程

第一步：获得Root 目录节点

第二步：解析路径，逐级通过lookup方法找到对应节点，直至目标节点

第三步：对目标节点执行操作

VfsOps::root_dir()



VfsNodeOps::lookup()



VfsNodeOps::op_xxx()

文件系统的示例 - Ext2

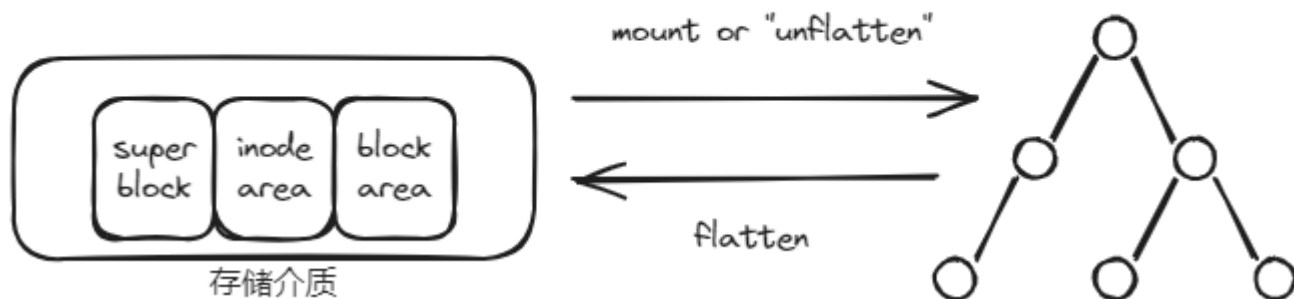


文件系统的mount - 意义1

mount可以理解为文件系统在内存中的展开操作（unflatten）
把易于存储的扁平化的形态转化为易于搜索遍历的立体化形态。

扁平结构 - 压扁的目录树

立体结构 - 展开的目录树



文件系统的mount - 意义2

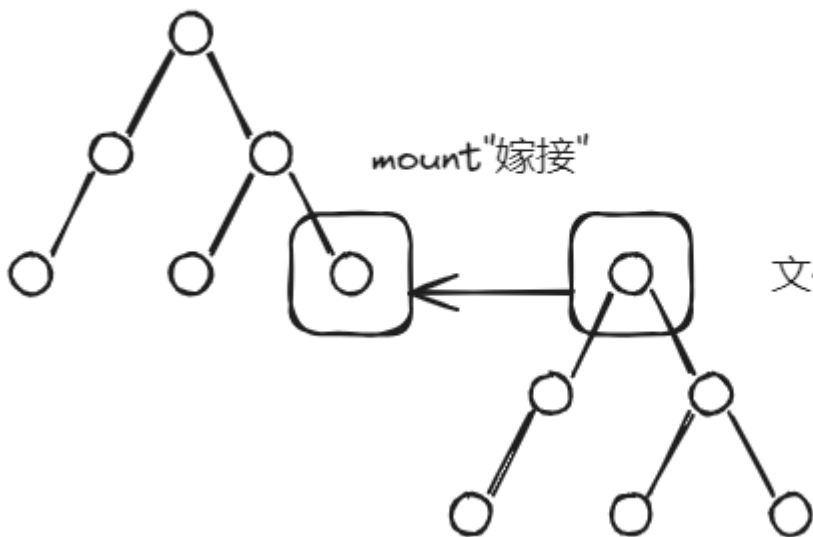
把一棵目录树的“根”“嫁接”到另一棵树的某个结点，两棵树就形成了一棵树。

两棵目录树基于的文件系统可以相同也可以不同。

另外，被mount的结点及其子孙结点都会被遮蔽，直至unmount。

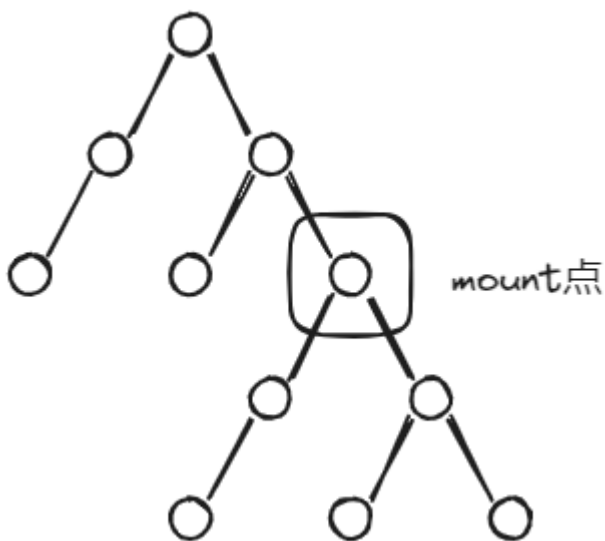
lookup操作到达mount点时，将会发生访问目录树的切换。

文件系统1展开的目录树



文件系统2展开的目录树

在mount点上lookup的特殊处理



```
struct RootDirectory {  
    main_fs: Arc<dyn VfsOps>,  
    mounts: Vec<MountPoint>,  
}
```

```
pub fn mount(&mut self, path: &'static str, fs: Arc<dyn VfsOps>) -> AxResult {  
    if path == "/" {  
        return ax_err!(InvalidInput, "cannot mount root filesystem");  
    }  
    if !path.starts_with('/') {  
        return ax_err!(InvalidInput, "mount path must start with '/'");  
    }  
    if self.mounts.iter().any(|mp| mp.path == path) {  
        return ax_err!(InvalidInput, "mount point already exists");  
    }  
    // create the mount point in the main filesystem if it does not exist  
    self.main_fs.root_dir().create(path, FileType::Dir)?;  
    fs.mount(path, self.main_fs.root_dir().lookup(path)?)?;  
    self.mounts.push(MountPoint::new(path, fs));  
    Ok(())  
}
```


在mount点上lookup的特殊处理

```
fn lookup_mounted_fs<F, T>(&self, path: &str, f: F) -> AxResult<T>
where
    F: FnOnce(Arc<dyn VfsOps>, &str) -> AxResult<T>,
{
    let mut idx = 0;
    let mut max_len = 0;

    // Find the filesystem that has the longest mounted path match
    // TODO: more efficient, e.g. trie
    for (i, mp) in self.mounts.iter().enumerate() {
        // skip the first '/'
        if path.starts_with(&mp.path[1..]) && mp.path.len() - 1 > max_len {
            max_len = mp.path.len() - 1;
            idx = i;
        }
    }

    if max_len == 0 {
        f(self.main_fs.clone(), path) // not matched any mount point
    } else {
        f(self.mounts[idx].fs.clone(), &path[max_len..]) // matched at `idx`
    }
}
```

课后练习 - 为RamFS支持rename操作

[ramfs_rename]:支持ramfs的rename操作。

预备:

执行**make run A=exercises/ramfs_rename/ BLK=y**

显示ramfs对rename操作不支持, 如下图:

```
Create '/tmp/f1' and write [hello] ...
Rename '/tmp/f1' to '/tmp/f2' ...
[ 0.586387 0 axfs_vfs:160] [AxError::Unsupported]
[ 0.590502 0 axruntime::lang_items:5] panicked at exercises/ramfs_rename/src/main.rs:51:9:
Error: Operation not supported
cloud@server:~/gitWork/oscamp/arceos$
```

要求:

1. 采用patch方式让工程临时使用oscamp/arceos/axfs_ramfs的本地组件仓库。
2. 修改本地组件axfs_ramfs, 增加相关函数, 实现部分trait, 让测试通过。

```
Create '/tmp/f1' and write [hello] ...
Rename '/tmp/f1' to '/tmp/f2' ...
Read '/tmp/f2' content: [hello] ok!

[Ramfs-Rename]: ok!
```

提示: 对第1点patch的方式, 参考

<https://rustwiki.org/zh-CN/edition-guide/rust-2018/cargo-and-crates-io/replacing-dependencies-with-patch.html>